Módulo 6 — Ambiente, Higiene y Servicios Urbanos

Módulo operativo de campo de la higiene urbana y el control ambiental. Planifica y ejecuta los servicios de recolección, limpieza, arbolado y espacios verdes; administra el inventario urbano ambiental; y tramita las denuncias ambientales hasta el acta de constatación, registrando después la resolución sancionatoria que devuelve M4.

Estado al 11 de noviembre de 2026 (entrega final).

Índice

1. Qué hace el módulo

2. Glosario

3. Funcionalidades

- Configuración de catálogos
- Programación de servicios
- Ejecución en campo
- Residuos
- Limpieza viaria
- Contenedores y puntos verdes
- Arbolado
- Espacios verdes
- Control ambiental
- Seguimiento del ciudadano
- Tablero de indicadores

4. Entidades y estados

- 4.1 El concepto central: `Service`
- 4.2 Entidades
- 4.3 Estados y transiciones

5. Eventos que publicamos

- 5.1 Qué información lleva cada evento
- 5.2 Los nueve que no publicamos
- 5.3 Enumeraciones

6. Eventos y datos que necesitamos recibir

- 6.1 Campos imprescindibles

7. Puntos bloqueantes

1. Qué hace el módulo

Área	Responsabilidad
Configuración de catálogos	Tipos de servicio, zonas operativas, frecuencias y recorridos
Programación de servicios	Agenda sobre recorridos u objetivos puntuales; reprogramación y cancelación
Ejecución en campo	Inicio, avance, suspensión y cierre; registro de lo que quedó sin atender y por qué
Residuos	Recolección domiciliaria, de reciclables, de voluminosos y de verdes; volumen, peso y destino final
Limpieza viaria	Barrido manual y mecánico, y lavado de calles, como servicio programable sobre recorrido
Contenedores y puntos verdes	Alta, ubicación, capacidad, desborde, daño, vaciado, reparación y reubicación. Puntos verdes de entrega voluntaria
Arbolado	Censo con historial de relevamientos; poda, extracción, plantación y tratamiento
Espacios verdes	Plazas y parques, con riego y corte de césped
Control ambiental	Denuncias ambientales —ruidos, vertidos, microbasurales, emisiones—, inspección, acta de constatación y violación constatada
Seguimiento del ciudadano	Publicamos el avance de la inspección para que M2 lo muestre, y servimos sin token una consulta pública complementaria: seguimiento de la denuncia, cuándo pasa el servicio y puntos verdes

2. Glosario

Término	Qué es en M6
Zona operativa (`Zone`)	Agrupación propia que junta uno o más barrios para armar recorridos y asignar cuadrillas. Hoy hay una por barrio del catálogo de M9 (48 barrios, 15 comunas)
Servicio (`Service`)	Unidad de trabajo programable: tipo, fecha, cuadrilla y estado. Puede ser `ROUTE` o `POINT`
Denuncia	El reclamo que el vecino presenta en M2, dueño del registro. En el vocabulario de M2 es un **ticket**, y ese es el nombre que usamos en eventos y campos (`ticketId`)
Acta (`ViolationNotice`)	Lo que emitimos al constatar una violación. Inmutable una vez emitida
Cuadrilla (`Crew`)	Equipo de trabajo, municipal o de cooperativa, que ejecuta servicios

3. Funcionalidades

Configuración de catálogos

- → Alta, edición y baja de tipos de servicio, zonas operativas y recorridos.
- → Definición de frecuencias, cuadrillas, vehículos y sitios de disposición final.

Programación de servicios

- → Reglas de frecuencia por recorrido, tipo de servicio y turno, con vigencia y sin solapamientos. La agenda se programa servicio por servicio: la generación automática para un rango de fechas no se implementó.
- → Servicios puntuales sobre un bien del inventario o una ubicación, manuales o derivados de un reclamo.
- → Asignación de zona a toda programación: se copia del recorrido, se toma del bien, o se deduce del barrio cuando el objetivo es una dirección suelta.
- → Asignación de cuadrilla y vehículo, con validación de disponibilidad y de no superposición.
- → Reprogramación individual y masiva —esta última ante alerta meteorológica— y cancelación, ambas con motivo obligatorio.
- → Vista de agenda por cuadrilla, zona y tipo de servicio.

Ejecución en campo

- → Inicio, avance, suspensión, reanudación y cierre del servicio.
- → Resultado por zona en los recorridos, o resultado único en los servicios puntuales.
- → Registro de lo no atendido, con motivo obligatorio y propuesta de nueva fecha.
- → Adjuntos y notas a nivel de servicio y de zona.
- → Aviso de demora —inicio tardío o duración excedida— con historial. No es un estado: el servicio sigue en `SCHEDULED` o `IN_PROGRESS`. El marcado automático por tolerancia no se implementó.

Residuos

- → Gestión diferenciada de residuos domiciliarios, reciclables, voluminosos y verdes.
- → Registro de volumen, peso y sitio de disposición al cerrar el servicio.
- → Retiros puntuales de voluminosos originados en un reclamo.
- → Consulta histórica por zona, tipo de residuo, destino y período.

Limpieza viaria

- → Barrido manual, barrido mecánico y lavado de calles como servicio recurrente sobre recorrido.
- → Mismo ciclo de ejecución y mismo resultado por zona que el resto de los servicios: no es un flujo aparte.

Contenedores y puntos verdes

- → Alta, edición y baja de contenedores, con código, tipo, ubicación, capacidad, zona y estado.
- → Registro de desborde y de daño, con tipo de daño y severidad.
- → Programación de vaciado, limpieza, reparación y reubicación.
- → Puntos verdes de entrega voluntaria, con materiales aceptados y estado, y retiro programado de lo acopiado hacia planta de reciclaje.
- → Listado por zona, con estado actual e historial.

Arbolado

- → Censo de árboles con especie, ubicación, altura y diámetro.
- → Relevamientos con estado sanitario y nivel de riesgo, conservando el historial completo.
- → Cuando el riesgo es alto o crítico el relevamiento avisa a M3 (`treeRiskDetected`); la intervención la solicita Oficina, no se genera sola.
- → Poda de formación, poda de seguridad, extracción, plantación y tratamiento.
- → La extracción exige autorización previa de un supervisor, con justificación.
- → Indicación de necesidad de corte de calle; la solicitud a M7 se pide aparte, sobre la intervención autorizada.

Espacios verdes

- → Alta, edición y baja de plazas, parques, canteros y espacios similares, con superficie, zona y estado.
- → Riego y corte de césped como servicio recurrente.
- → Lo detectado durante un servicio o una inspección se deriva a M3 como solicitud de reparación. No hay un registro de incidencias propio de los espacios verdes.

Control ambiental

- → Registro del expediente técnico a partir de una denuncia derivada de M2 o de una detección de oficio.
- → Priorización, y elevación de prioridad ante escalamiento del reclamo.
- → Pedido de información al vecino durante la revisión, que sale hacia M2 como `INFORMATION_REQUIRED` .
- → Inspecciones con checklist, hallazgos, adjuntos y resultado.
- → Emisión del acta de constatación, con numeración correlativa e inmutable una vez emitida. Solo se deriva a M4 si identificamos el establecimiento.
- → Registro de la resolución sancionatoria que devuelve M4 —multa, clausura o desestimación—, que es lo que cierra el expediente.
- → Desestimación con motivo, y cierre con resolución.

Seguimiento del ciudadano

- → Publicación del avance de la inspección hacia M2, que es el canal oficial del vecino. Nuestra vista es complementaria: se alimenta de los mismos eventos, así que no pueden divergir.
- → Vista de consulta pública, sin token y de solo lectura: seguimiento de la denuncia por el `ticketId` del reclamo, resumido en siete etapas y con la fecha de inspección; cuándo pasa el servicio por una zona; y puntos verdes activos. Tiene límite de consultas por minuto.
- → **No calculamos SLA**: el plazo comprometido con el vecino lo fija M2.
- → Proyección pública restringida, tanto en la vista como en lo que publicamos por evento: nunca notas internas, contenido del acta, identidad del inspector ni datos de terceros.
- → No hay listado de denuncias por ciudadano ni rol de ciudadano en M6. El ciudadano no crea, modifica ni cierra nada en este módulo.

Tablero de indicadores

- → **Cobertura**: objetivos atendidos sobre programados, por período, zona y tipo de servicio.
- → **Cumplimiento**: servicios finalizados en término, demorados, y ranking de zonas no atendidas con sus motivos.
- → **Incidencias**: contenedores desbordados y dañados por zona, árboles por nivel de riesgo, denuncias por tipo y estado, y tiempo medio de resolución.
- → **Residuos**: kilos y metros cúbicos por tipo y destino, y porcentaje desviado a reciclaje.

4. Entidades y estados

4.1 El concepto central: `Service`

Un `Service` es toda unidad de trabajo programable del módulo: siempre tiene tipo, fecha, ventana horaria, cuadrilla, vehículo si corresponde, estado y evidencia. Lo que varía es sobre qué se ejecuta.

`mode`	Objetivo	Ejemplo	Resultado
`ROUTE`	Secuencia ordenada de zonas	Recolección domiciliaria Zona Norte	Un `ZoneResult` por zona
`POINT`	Un bien del inventario o una ubicación	Vaciar el contenedor CT-0442; podar el árbol TR-01293	Un único resultado

Todo `Service` lleva `zoneIds[]` y nunca viene vacío: las zonas del recorrido en `ROUTE`, una sola en `POINT`. Se copia al programar y no se recalcula, para que editar un recorrido no altere lo ya ejecutado.

En `POINT` la zona sale del bien del inventario, o del barrio de la dirección si el objetivo es una ubicación suelta.

Las podas y las inspecciones ambientales también son servicios: `TreeIntervention` y `EnvironmentalInspection` guardan qué hay que hacer, y el `Service` cuándo, con qué cuadrilla y cómo terminó.

4.2 Entidades

Configuración y planificación

Entidad	Campos principales
`ServiceType`	`code`, `name`, `category`, `mode`, `requiresVehicle`
`Zone`	Zona **operativa** propia, hoy una por barrio del catálogo de M9. `code`, `name`, `communeId?`, `neighborhoodIds[]`, `active`
`Commune`	Comuna de M9 (15) que agrupa zonas. `code`, `number`, `name`, `active`
`Route`	`code`, `name`, y la secuencia de `RouteStop` (`sequence`, `zoneId`, `estimatedDurationMin`)
`ServiceFrequency`	`serviceTypeId`, `routeId`, `weekdays[]`, `shift`, `validFrom`, `validTo`

Recursos

Entidad	Campos principales
`Crew`	`name`, `crewType`, `specialty`, `homeZoneId`, `leaderUserId`, `memberUserIds[]`, `organizationId`, `defaultShift`, `active`
`Vehicle`	`plate`, `vehicleType`, `capacity`, `active`

Ejecución

Entidad	Campos principales
`Service`	`serviceTypeId`, `mode`, `zoneIds[]`, `routeId` | `targetRef`, `scheduledDate`, `timeWindow`, `crewId`, `vehicleId`, `status`, `statusReason`, `origin`, `ticketId`, `weatherAlertId`, `attachments[]`, `notes`. `crewId` es opcional hasta que se asigna la cuadrilla. Cada aviso de demora queda en un `ServiceDelayNotice` hijo
`ZoneResult`	Hijo de un `Service` con `mode = ROUTE`, uno por zona. `zoneId`, `status`, `reason`, `proposedDate`, `notes`, `attachments[]`, `recordedAt`
`CollectionRecord`	`wasteType`, `volumeM3`, `weightKg`, `disposalSiteId`
`DisposalSite`	`code`, `siteType`, `name`

Inventario urbano

Entidad	Campos principales
`Container`	`code`, `containerType`, `location`, `capacityLiters`, `zoneId`, `status`, y con el estado `DAMAGED` también `damageType`, `severity`, `requiresPublicWorks`
`GreenPoint`	Punto verde de entrega voluntaria. `code`, `name`, `location`, `acceptedWasteTypes[]`, `zoneId`, `active`
`Tree`	`surveyCode`, `species`, `location`, `heightM`, `diameterCm`, `zoneId`, más el historial de `TreeSurvey`
`TreeSurvey`	`treeId`, `surveyedAt`, `inspectorId`, `healthStatus`, `riskLevel`, `riskType`, `suggestedIntervention`, `requiresStreetClosure`, `requiresPublicWorks`, `notes`
`TreeIntervention`	`interventionType`, `treeIds[]`, `location`, `requiresStreetClosure`, `status`, `priority`, `serviceId`, y en las extracciones `authorizedByUserId`, `authorizedAt`, `justification`
`GreenSpace`	`name`, `spaceType`, `lat`, `lng` (opcionales), `areaM2`, `zoneId`, `active`

Control ambiental

Entidad	Campos principales
`EnvironmentalReport`	`reportType`, `location`, `description`, `caseNumber`, `ticketId`, `publicId`, `reporterSnapshot`, `status`, `priority`, `escalated`, `citizenResponse`, `deadlineAt`
`EnvironmentalInspection`	`reportId`, `serviceId`, `inspectorId`, `inspectedAt`, `checklist[]`, `findings`, `attachments[]`, `outcome`, `conclusion`, `violationType`, `severity`, `suggestedAction`
`ViolationNotice`	El acta. `noticeNumber`, `inspectionId`, `issuedAt`, `establishmentId`, `violationType`, `severity`, `suggestedAction`, `priorNoticeCount`. Inmutable
`SanctionOutcome`	La resolución de M4, espejada de solo lectura para poder cerrar el expediente. `violationNoticeId`, `decision`, `decidedAt`, `externalRef`, `dismissalReason`. M4 confirmó (24/08) que `decidedAt` y `externalRef` vuelven a su payload

Derivaciones salientes

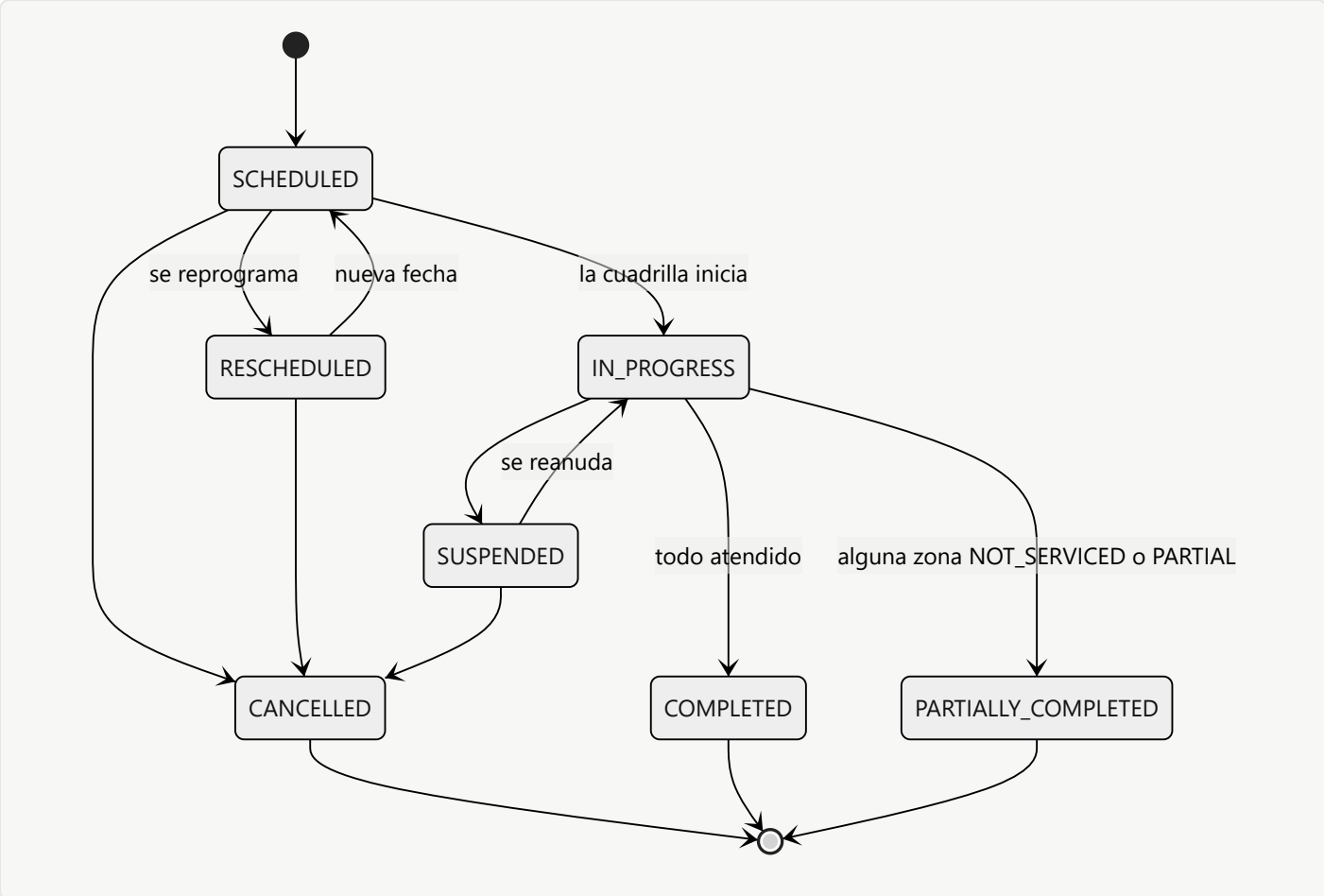
Entidad	Campos principales
`RepairRequest`	Hacia M3. `damageType`, `location`, `severity`, `publicSafetyRisk`, `detectedIn`, `status`, `workOrderId`. Tres estados: `REQUESTED`, `IN_PROGRESS`, `CLOSED`

Entidad	Campos principales
`StreetClosureRequest`	Hacia M7. `reason`, `sourceRef`, `affectedSections[]` (tramos con calle, entre qué calles y `direction`), `requestedFrom`, `requestedTo`, `closureType`, `status` (`REQUESTED`, `APPROVED`, `REJECTED`, `ENDED`, `EXPIRED`)

4.3 Estados y transiciones

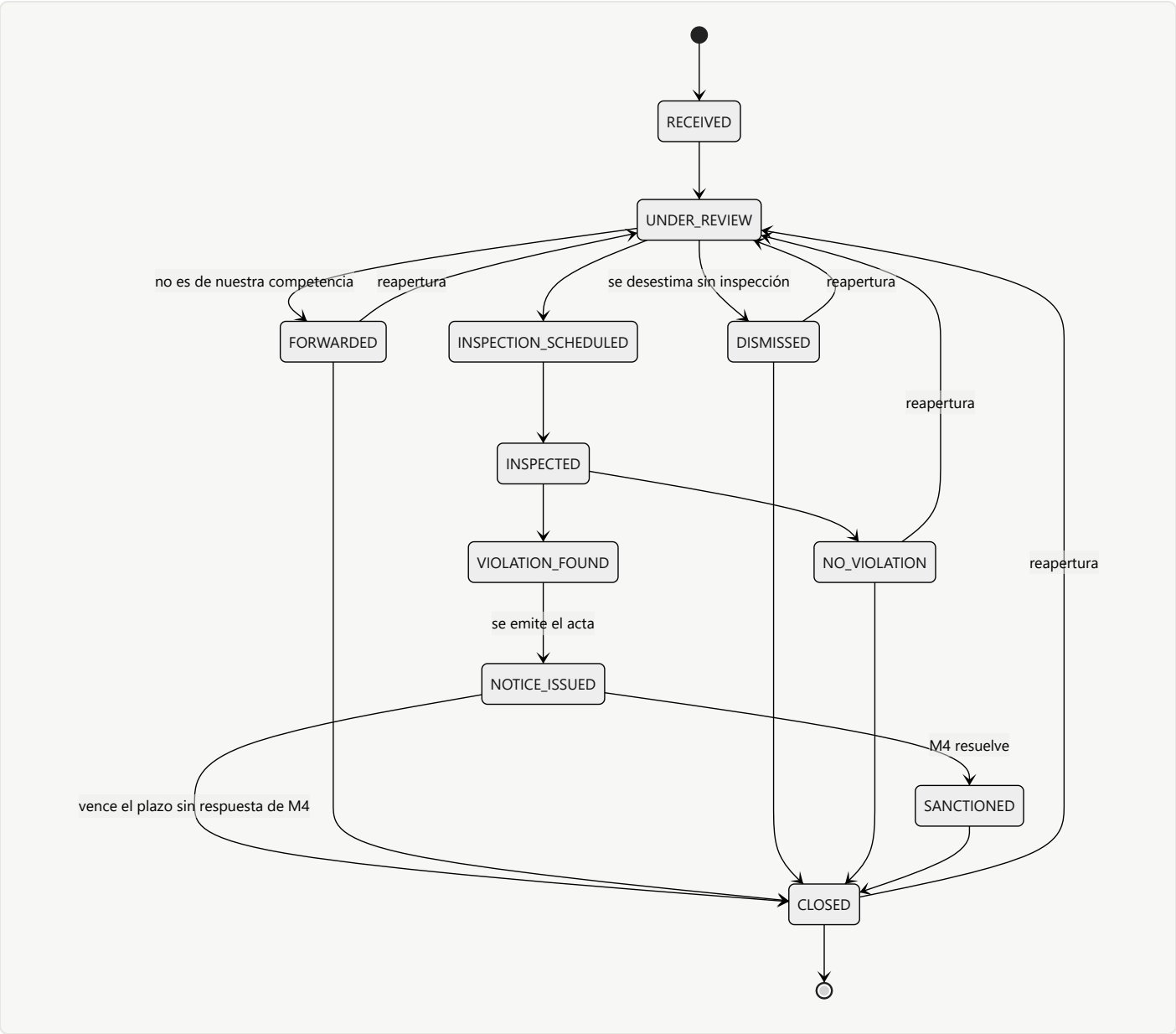
Estos son los estados que un módulo consumidor puede llegar a ver reflejados en nuestros eventos.

`Service` — el mismo diagrama vale para `mode = ROUTE` y para `mode = POINT`

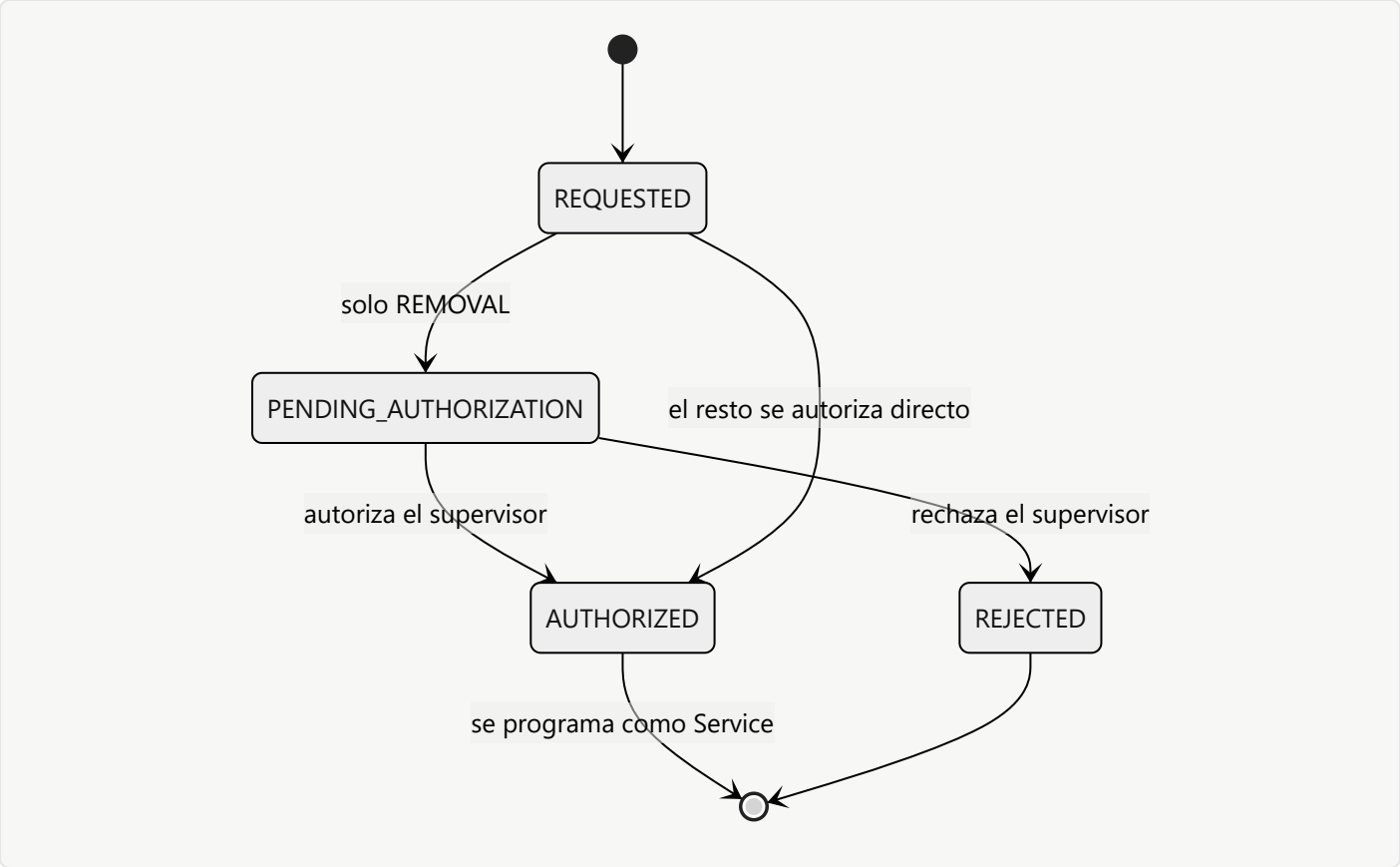


`DELAYED` no es un estado, es un aviso puntual: la demora queda registrada como un hecho del servicio (`urbanServiceDelayed`, que no sale al bus, ver 5.2) y el servicio sigue en `SCHEDULED` o `IN_PROGRESS`.

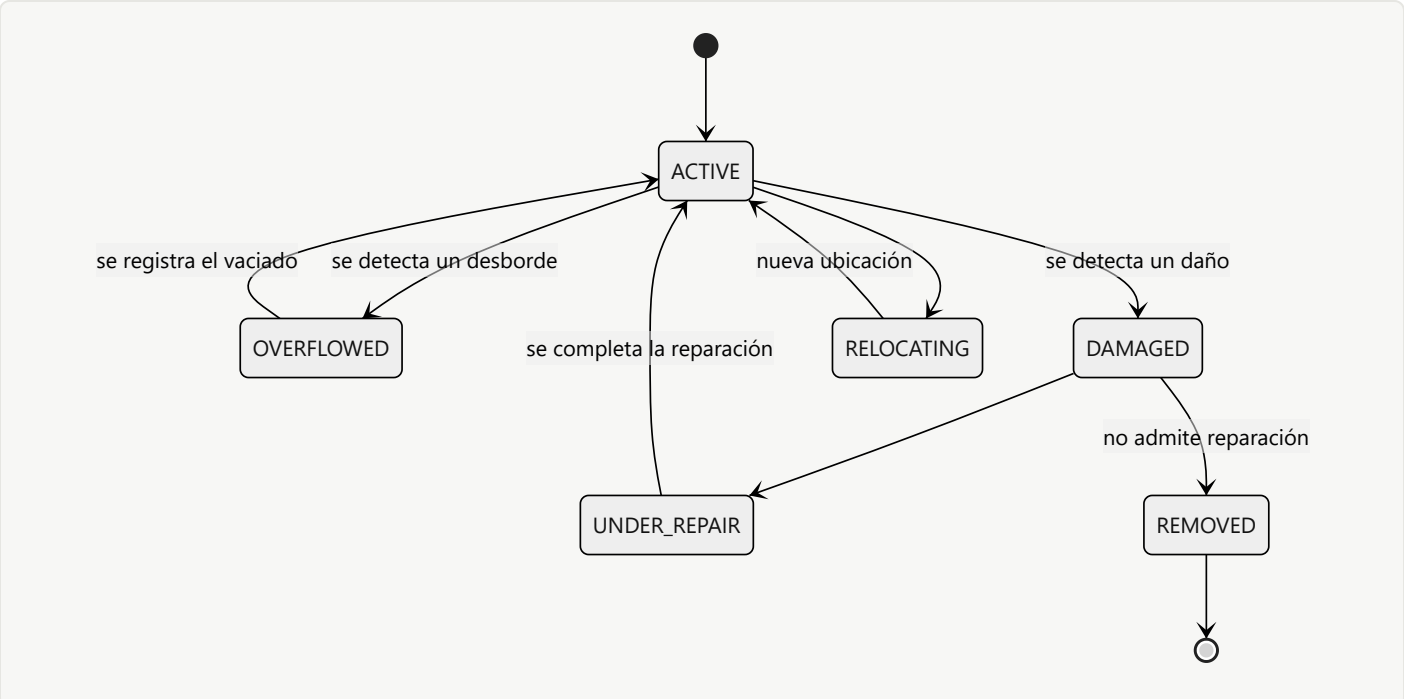
`EnvironmentalReport`



`TreeIntervention` — la extracción necesita autorización; el resto no



`Container`



5. Eventos que publicamos

Seis eventos, y los seis tienen consumidor confirmado.

Evento	Se dispara cuando	Consumidores
`updateTicketStatus`	Cambia el estado de un servicio o una inspección nacidos de un reclamo	**M2**
`containerDamaged`	Se detecta un contenedor dañado o faltante	**M3**
`treeRiskDetected`	Un relevamiento arroja riesgo `HIGH` o `CRITICAL`	**M3**
`environmentalViolationDetected`	Se emite un acta ambiental	**M4**
`infrastructureRepairRequested`	Detectamos un daño de infraestructura ajeno a nuestra competencia	**M3**
`streetClosureRequested`	Un servicio o intervención requiere cortar la calle	**M7**

Eventos que decidimos no publicar

Evento	Por qué no
`urbanServiceStarted`, `urbanServiceDelayed`, `urbanServiceCompleted`, `zoneNotServiced`, `containerOverflowed`, `environmentalInspectionScheduled`, `environmentalInspectionCompleted`	Estaban en el contrato porque se los ofrecíamos a M2 como detalle del avance de un reclamo. Cuando M2 definió que todo lo suyo entra por `updateTicketStatus`, se quedaron sin ningún consumidor. **Los hechos siguen existiendo en nuestro modelo** —son los que disparan cada `updateTicketStatus` — pero no salen al bus: un evento que nadie escucha es schema, publisher, test y documentación para nadie. El payload de cada uno está diseñado: si alguno hace falta, se pide y se publica
`urbanServiceScheduled`, `treePruningScheduled`	Se publicaban hacia M7 y **dejaron de emitirse el 21/10**: M7 confirmó que solo consume `streetClosureRequested`, así que se quedaron sin consumidor. La fecha agendada se informa a M2 como texto dentro de `updateTicketStatus`. El payload queda diseñado en 5.2
`environmentalFineIssued`	No generamos cargos económicos. Nuestra acta llega a M5 a través de M4, convertida en `commercialFineGenerated`
`environmentalReportCreated`	El dueño de la denuncia es M2; que abramos un expediente interno no le agrega nada. Lo que sí necesita es la inspección programada y su resultado, que publicamos
`citizenNotificationRequested`	Las notificaciones las resuelve M9. Nadie publica algo que las origine, y el 27/10 se descartó `notificationSent`: M6 no solicita notificaciones

5.1 Qué información lleva cada evento

Información de negocio, no formato: el sobre lo define el Core de M9 (contrato del 7/10) y lo usamos tal cual. Estos son los payloads implementados, y cada uno tiene su JSON Schema. `?` es opcional o puede venir en nulo, `[]` es lista. Los valores de cada enumeración están en 5.3.

`containerDamaged`

```
containerId, containerCode, zoneId, location,
damageType, severity, requiresPublicWorks,
detectedAt, ticketId?
```

`requiresPublicWorks = true` marca el componente de infraestructura civil ajeno a nosotros: es la señal para M3. Sale con `true` o `false`, y M3 procesa solo los `true`. Es un aviso sin `requestId`: no esperamos respuesta de M3.

`treeRiskDetected`

```
treeId, surveyCode, species, zoneId, location,
riskLevel, riskType, healthStatus,
suggestedIntervention?,
requiresStreetClosure, requiresPublicWorks,
surveyedAt
```

Solo se dispara con `riskLevel` en `HIGH` o `CRITICAL`. `healthStatus` es `HEALTHY`, `WEAKENED`, `DISEASED` o `DEAD`, y `suggestedIntervention` es opcional (monitorear no es una intervención: lo expresa su ausencia). M3 lo aceptó tal cual, así que queda descartado el evento genérico con `hazardType` que ofrecíamos como alternativa, y su alias `urbanRiskDetected` también. Es un aviso: M3 correlaciona por `treeId` y no responde.

`environmentalViolationDetected` → **M4**

```
violationId, noticeNumber, issuedAt,
reportId, inspectionId, ticketId?,
violationType, severity, location,
establishmentId,
priorNoticeCount,
evidence[], suggestedAction
```

`establishmentId` es obligatorio: es lo único sobre lo que M4 puede actuar, porque intimar, clausurar y multar se le aplican a un comercio habilitado. Si no identificamos el establecimiento no derivamos el acta y el expediente cierra de nuestro lado. `suggestedAction` **no es vinculante** y puede ser `WARNING`, `FORMAL_NOTICE`, `FINE` o `CLOSURE`; falta que M4 confirme que tolera `FORMAL_NOTICE`.

`infrastructureRepairRequested` → **M3**

```
requestId, damageType, severity, location,
detectedIn, ticketId?,
publicSafetyRisk, requestedAt
```

✅ Pedimos que M3 devuelva `requestId` como `sourceRequestId`, y ya lo confirmaron —en `workOrderScheduled` y en `workOrderCompleted`—. Es el único de los tres eventos hacia M3 con respuesta, y la solicitud se cierra con `workOrderCompleted`. `ticketId` es el nombre de la cohorte; M3 lo llamaba `complaintId`.

`streetClosureRequested` → **M7**

```
closureRequestId, sourceModule, sourceRef,
reason, affectedSections[], requestedFrom, requestedTo,
closureType?, requestedAt
```

`sourceRef` apunta al servicio o a la intervención que origina el corte. `sourceModule` es `M3` o `M6`: nosotros mandamos siempre `M6`. Es el esquema unificado con la solicitud de M3 que propuso M7 el 30/08 y aceptamos: antes eran `requestId`, `streets[]`, `from` y `to`. ✅ M7 devuelve el origen — `closureRequestId` y `requestingModule` (`"Ambiente"`, en nuestro caso) — en `streetClosureApproved` y `streetClosureRejected` desde el 25/08, y en `streetClosureEnded` desde el 30/08.

`updateTicketStatus` → M2

```
ticketId, updateType,
publicMessage?, internalMessage?, progress?, details?,
attachments[]?, updatedBy, updateOccurredAt
```

Es el canal del vecino, y el payload lo define M2 en su contrato, hoy en su **v1.78** (WIP; la v1.6 fue la última que cambió su forma, y la v1.78 acotó `updatedBy.id` a un UUID o nulo): lo adoptamos tal cual. Lo publicamos **solo cuando el servicio o la inspección nacieron de un reclamo**: un servicio planificado no genera nada hacia M2. No lleva estado —informamos el hecho con `updateType` y M2 decide la transición— ni `sourceRef`, porque su contrato prohíbe transportar IDs de nuestras entidades internas: la correlación `ticketId ↔ serviceId` queda de nuestro lado. 🔄 Desde la v1.2 salieron `publicId` y `expectedTicketVersion` del evento (la correlación es solo por `ticketId`), `message` se partió en `publicMessage` / `internalMessage`, y la v1.6 renombró la fecha a `updateOccurredAt`, cerró `updatedBy` en `EXTERNAL_USER` (una persona nuestra) o `SYSTEM` (un proceso automático, con `id` nulo), sacó `attachmentId` del adjunto y cambió `resolution.type` `INFORMATION_PROVIDED` por `INQUIRY_ANSWERED`. El sobre es el del Core, sin campos propios: la correlación con M2 va por `ticketId`.

Estos son los hechos internos que lo disparan:

`updateType`	Se publica cuando	`details`
`STARTED`	Se agenda o se inicia un servicio, o se programa la inspección, mientras el ticket todavía no está en curso en M2 (agendar cuenta como tomar la gestión)	`publicMessage` con la fecha y la franja como texto; sin `details`
`PROGRESS`	Lo mismo con el ticket ya en curso: reprogramación, inicio, demora (solo con el servicio iniciado) o emisión del acta. `zoneNotServiced` está diseñado y sin emitir	`publicMessage` o `internalMessage`. La fecha agendada va como texto: M2 lo confirmó el 20/10
`RESOLVED`	`urbanServiceCompleted`, o cierra la inspección sin irregularidad	`resolution.type` + `publicMessage`
`RETURNED`	El reclamo no es de nuestra área. Vuelve a M2 para que lo re-derive	`returnInfo.reasonCode`
`REJECTED`	El expediente pasa a `DISMISSED`	`cancellation.reasonCode`

`updateType`	Se publica cuando	`details`
`INFORMATION_REQUIRED`	El inspector necesita un dato del vecino (`POST /environmental-reports/:id/request-information`)	`informationRequest.messageForCitizen` (+ `requiredBy` opcional). Emitido. No hay `informationRequestId` : M2 admite una sola solicitud de información activa por ticket

Publicamos seis eventos y los seis tienen consumidor declarado. Eran quince: los siete que le ofrecíamos a M2 como detalle del avance — `urbanServiceStarted` , `urbanServiceDelayed` , `urbanServiceCompleted` , `zoneNotServiced` , `containerOverflowed` , `environmentalInspectionScheduled` y `environmentalInspectionCompleted` — se quedaron sin consumidor cuando M2 definió que todo lo suyo entra por `updateTicketStatus` , y los sacamos del contrato. Después, el 21/10, `urbanServiceScheduled` y `treePruningScheduled` dejaron de emitirse: M7 no los consume. Los hechos siguen existiendo en nuestro modelo y son los que disparan cada `updateTicketStatus` , pero no salen al bus: un evento que nadie escucha es código que hay que escribir, versionar y mantener para nadie. Si alguno hace falta, se pide y se publica.

5.2 Los nueve que no publicamos

El diseño queda escrito. Siete son los hechos que disparan cada `updateTicketStatus` ; los otros dos, `urbanServiceScheduled` y `treePruningScheduled` , salían hacia M7 y dejaron de emitirse el 21/10. Si algún módulo los pide, se publican sin rediseñar nada.

`urbanServiceScheduled` → **M7** *Dejó de emitirse el 21/10: M7 no lo consume.*

```
serviceId, serviceTypeCode, category, mode,
zoneIds[], routeId?, targetRef?,
scheduledDate, timeWindow { from, to },
crewId?, vehicleId?,
origin, ticketId?
```

`zoneIds[]` : una sola zona en `POINT` , las del recorrido en `ROUTE` . `targetRef` : el bien del inventario, solo en `POINT` . `ticketId` : viaja siempre que `origin = TICKET` .

`treePruningScheduled` → **M7** *Dejó de emitirse el 21/10: M7 no lo consume.*

```
interventionId, serviceId, interventionType, treeIds[],
zoneId, location,
scheduledDate, timeWindow { from, to },
crewId, requiresStreetClosure
```

La fecha, la ventana y la cuadrilla salen del `Service` , que va identificado por si quieren seguirlo con `urbanServiceScheduled` . Como `crewId` y `timeWindow` eran requeridos para M7 y opcionales en nuestro modelo, **el evento solo salía cuando el servicio ya tenía cuadrilla y franja horaria**.

`urbanServiceStarted`

```
serviceId, serviceTypeCode, mode,
startedAt, crewId, zoneIds[]
```

`urbanServiceDelayed`

```
serviceId, delayType, delayMinutes,
reason, newEstimatedEnd, detectedAt
```

Se publica una sola vez por servicio.

`urbanServiceCompleted`

```
serviceId, serviceTypeCode, mode, outcome,
startedAt, completedAt,
zoneResults[] { zoneId, status, reason? },
collection? { wasteType, volumeM3, weightKg, disposalSiteId },
attachments[], notes?, ticketId?
```

`zoneResults[]` solo en `mode = ROUTE`; `collection` solo en los servicios de recolección.

`zoneNotServiced`

```
serviceId, zoneId, neighborhoodIds[],
reason, notes?, proposedDate?, recordedAt
```

`neighborhoodIds[]` va explícito para que M2 ubique a los vecinos afectados sin conocer nuestras zonas.

`containerOverflowed`

```
containerId, containerCode, containerType,
zoneId, location,
detectedAt, ticketId?
```

`environmentalInspectionScheduled` → **M2**

```
inspectionId, serviceId, reportId, ticketId?,
reportType, zoneId, location,
scheduledDate, timeWindow { from, to }
```

`environmentalInspectionCompleted` → **M2**

```
inspectionId, reportId, ticketId?,
inspectedAt, outcome, publicSummary, nextStep
```

Los dos llevan la **proyección pública**: sin identidad del inspector, checklist, hallazgos internos ni contenido del acta. `publicSummary` es el texto que lee el vecino, y `nextStep` dice qué sigue —acta, reinspección o cierre— sin adelantar la sanción.

5.3 Enumeraciones

Son las materializadas en la base. Donde difieren de lo que circuló en el primer acuerdo de eventos, manda este catálogo (ADR-003): `ServiceOrigin` tiene cinco valores, `TreeHealthStatus` conserva `WEAKENED` y `DISEASED`, `TreeInterventionType` distingue las dos podas y usa `REMOVAL`, y `SuggestedAction` incluye `FORMAL_NOTICE`. `DamageType` y `ViolationType` viajan como texto en los eventos.

Enumeración	Valores
`ServiceCategory`	`WASTE_COLLECTION`, `STREET_CLEANING`, `CONTAINERS`, `TREES`, `GREEN_SPACES`, `ENVIRONMENTAL_CONTROL`
`ServiceMode`	`ROUTE`, `POINT`
`ServiceStatus`	`SCHEDULED`, `RESCHEDULED`, `IN_PROGRESS`, `SUSPENDED`, `COMPLETED`, `PARTIALLY_COMPLETED`, `CANCELLED`
`ServiceOrigin`	`PLANNED`, `TICKET`, `WEATHER_ALERT`, `INSPECTION`, `MANUAL`
`DelayType`	`START`, `DURATION`
`ZoneResultStatus`	`SERVICED`, `PARTIAL`, `NOT_SERVICED`
`NotServicedReason`	`VEHICLE_BREAKDOWN`, `CREW_UNAVAILABLE`, `BLOCKED_ACCESS`, `STREET_CLOSURE`, `WEATHER`, `EXCESS_VOLUME`, `SECURITY_INCIDENT`, `OTHER`
`WasteType`	`HOUSEHOLD`, `RECYCLABLE`, `BULKY`, `GREEN`, `MIXED`
`DisposalSiteType`	`LANDFILL`, `TRANSFER_STATION`, `RECYCLING_PLANT`, `COMPOSTING_PLANT`
`ContainerType`	`HOUSEHOLD`, `RECYCLABLE`, `BULKY`, `GREEN`
`ContainerStatus`	`ACTIVE`, `OVERFLOWED`, `DAMAGED`, `UNDER_REPAIR`, `RELOCATING`, `REMOVED`
`DamageType`	`STRUCTURAL`, `BURNT`, `LID_BROKEN`, `WHEELS_BROKEN`, `VANDALIZED`, `MISSING`
`Severity`	`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`
`TreeHealthStatus`	`HEALTHY`, `WEAKENED`, `DISEASED`, `DEAD`
`RiskLevel`	`NONE`, `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`
`RiskType`	`FALLING_BRANCH`, `TRUNK_INSTABILITY`, `ROOT_UPLIFT`, `POWER_LINE_CONTACT`, `SIGN_OBSTRUCTION`, `PEST_INFESTATION`
`TreeInterventionType`	`FORMATION_PRUNING`, `SAFETY_PRUNING`, `REMOVAL`, `PLANTING`, `TREATMENT`
`TreeInterventionStatus`	`REQUESTED`, `PENDING_AUTHORIZATION`, `AUTHORIZED`, `REJECTED`
`GreenSpaceType`	`SQUARE`, `PARK`, `PLANTER`, `MEDIAN`, `PROMENADE`
`EnvironmentalReportType`	`NOISE`, `DUMPING`, `ILLEGAL_DUMPSITE`, `WATER_DISCHARGE`, `AIR_EMISSION`, `ODOR`, `PEST_INFESTATION`, `OTHER`
`EnvironmentalReportStatus`	`RECEIVED`, `UNDER_REVIEW`, `FORWARDED`, `DISMISSED`, `INSPECTION_SCHEDULED`, `INSPECTED`, `NO_VIOLATION`, `VIOLATION_FOUND`, `NOTICE_ISSUED`, `SANCTIONED`, `CLOSED`
`InspectionOutcome`	`NO_VIOLATION`, `VIOLATION_FOUND`, `INCONCLUSIVE`
`InspectionNextStep`	`NOTICE_TO_BE_ISSUED`, `REINSPECTION`, `CASE_CLOSED`
`ViolationType`	`NOISE_LIMIT`, `ILLEGAL_DUMPING`, `UNTREATED_DISCHARGE`, `HAZARDOUS_WASTE`, `AIR_EMISSION`, `NO_WASTE_MANAGEMENT`, `INSPECTION_OBSTRUCTION`
`SuggestedAction`	`WARNING`, `FORMAL_NOTICE`, `FINE`, `CLOSURE`

Enumeración	Valores
`SanctionDecision`	`FINE_ISSUED`, `CLOSURE_ORDERED`, `FORMAL_NOTICE_ISSUED`, `DISMISSED`
`CrewType`	`MUNICIPAL`, `COOPERATIVE`, `CONTRACTOR`
`VehicleType`	`COMPACTOR_TRUCK`, `DUMP_TRUCK`, `SWEEPER`, `WATER_TANKER`, `CRANE_TRUCK`, `VAN`
`CrewSpecialty`	`COLLECTION`, `SWEEPING`, `CLEANING`, `RECYCLING`, `TREES`, `GREEN_SPACES`, `INSPECTION`
`Shift`	`MORNING`, `AFTERNOON`, `NIGHT`
`RepairDamageType`	`BROKEN_PAVEMENT`, `BROKEN_SIDEWALK`, `BROKEN_STREETLIGHT`, `BLOCKED_DRAIN`, `DAMAGED_STRUCTURE`
`RepairRequestStatus`	`REQUESTED`, `IN_PROGRESS`, `CLOSED`
`StreetClosureType`	`TOTAL`, `PARTIAL`
`StreetDirection`	`FORWARD`, `REVERSE`, `BOTH`
`StreetClosureRequestStatus`	`REQUESTED`, `APPROVED`, `REJECTED`, `ENDED`, `EXPIRED`

El vocabulario de `updateType` de `updateTicketStatus` (`STARTED`, `PROGRESS`, `INFORMATION_REQUIRED`, `RETURNED`, `RESOLVED`, `REJECTED`) es de M2 y no se replica como enum propio.

6. Eventos y datos que necesitamos recibir

Evento	Origen	Uso
`ticketUpdated`	M2	Según el `updateType`: `ROUTED` abre el expediente ambiental (nunca un servicio directo), y `INFORMATION_PROVIDED`, `CANCELLED`, `REOPENED`, `PRIORITY_CHANGED` y `ESCALATION_CHANGED` lo hacen avanzar. Los otros siete se descartan a propósito
`workOrderScheduled`	M3	Pasamos la solicitud de reparación a `IN_PROGRESS`
`workOrderCompleted`	M3	Cerramos la solicitud de reparación asociada
`commercialFineGenerated`	M4	Registramos la resolución del acta, pasamos a `SANCTIONED` y cerramos
`closureUpdate` (`status: ORDERED`)	M4	Ídem, con la clausura como resolución
`closureUpdate` (`status: LIFTED`)	M4	Registramos el levantamiento de la clausura y cerramos el expediente
`streetClosureApproved`	M7	Habilitamos la ejecución del servicio bloqueado
`streetClosureRejected`	M7	Reprogramamos o cancelamos el servicio dependiente
`streetClosureEnded`	M7	Liberamos la dependencia y habilitamos la reprogramación
`weatherAlertIssued`	Simulada por nosotros — nadie la publica hoy	Con severidad `HIGH` o `CRITICAL`, marcamos para reprogramar los servicios de las zonas afectadas dentro de la ventana de la alerta
`notificationSent`	M9	Sin handler ni suscripción: nadie lo publica y el 27/10 se descartó. Queda documentado, pero M6 no lo espera

6.1 Campos imprescindibles

Solo lo que necesitamos sí o sí. Cualquier campo adicional que ya publiquen lo aprovechamos, pero sin estos el flujo no funciona.

Origen	Evento o consulta	Campos imprescindibles
M2	`ticketUpdated / ROUTED`	Comunes: `ticketId`, `citizenId`, `isAnonymous`, `responsibleAreaId` ✅ (nuestro identificador es `M6`). En `details.routing`: `requestType` (string, sin IDs), `summary`, `description`, `location { street, streetNumber, neighborhoodId }`, sin coordenadas (v1.6 a v1.78)
M1	Consulta de empleado *(REST, `GET /integraciones/v1/actores`)*	`actorId`, `estado`, `nombre`, `apellido`. Se usa para validar al líder y los integrantes de una cuadrilla
M1	Consulta de ciudadano *(REST)*	`citizenId`, `fullName`, `documentId`. Hoy sin consumo
M1	Consulta de organización *(REST)*	`organizationId`, `legalName`. Hoy sin consumo
M3	`workOrderScheduled`	`sourceRequestId` ✅

Origen	Evento o consulta	Campos imprescindibles
M3	`workOrderCompleted`	`sourceRequestId` ✅. `outcome` (texto libre) y `attachments[]` (forma de adjunto de M2) los aceptamos, pero no los interpretamos
M4	`commercialFineGenerated`, `closureUpdate` (`status: ORDERED | LIFTED`)	`sourceViolationId` ✅, `decision`, `decidedAt` 🟢 (confirmado 24/08)
M7	`streetClosureApproved`	`streetClosureId`, `closureRequestId` ✅, `requestingModule` ✅, `startDate`, `endDate`
M7	`streetClosureRejected`	`closureRequestId` ✅, `rejectionReason`, `requestingModule` ✅
M7	`streetClosureEnded`	`streetClosureId`, `closureRequestId` ✅, `completionDateTime`
M9	`notificationSent`	Referencia al evento que la originó

✅ **Lo de M2 que pedimos se cerró.** `responsibleAreaId`, `citizenId` e `isAnonymous` son campos comunes de `ticketUpdated`, y la ubicación trae `neighborhoodId`. La v1.6 movió `requestType`, `summary`, `description` y `location` a `details.routing`, y sacó las coordenadas: lo leemos de las dos formas.

✅ **Los de M7 están confirmados**, con nombres reales distintos a los tentativos que teníamos anotados (`closureRequestId` / `requestingModule`, no `sourceRequestId` / `sourceModule`). Desde el 30/08 las tres respuestas traen el origen, `streetClosureEnded` incluida.

✅ **El de M3 está cerrado (5 oct):** consumimos solo `workOrderScheduled` y `workOrderCompleted`, y la solicitud se cierra con `workOrderCompleted`.

✅ **El de M4 se cerró.** `sourceViolationId` en la resolución —el `violationId` que mandamos en el acta, sin el cual no sabíamos cuál resolvían— ya viaja en `commercialFineGenerated` y `closureUpdate`.

✅ **La fecha agendada ya no es un bloqueante con M2.** `progress` sigue siendo un `Int` (porcentaje) y `STARTED` / `PROGRESS` siguen sin `details`, pero el 20/10 M2 confirmó que la fecha y la franja van como texto en `publicMessage`, y así sale desde el 21/10.

7. Puntos bloqueantes

Estado al 11 de noviembre de 2026. **Hoy no hay ningún punto bloqueante**: M9 habilitó el broker el 28/10 y M1 ya emite y valida el JWT. Quedan cinco puntos abiertos, todos de espera y ninguno frena el trabajo del módulo. El detalle por contraparte, con la fecha de cada pedido, está en `docs/bloqueantes.md` del repositorio del backend: si difieren, vale ese.

Abiertos

Punto	Con quién	Estado
Schema único de `streetClosureRequested`. M3 y M6 publican el mismo nombre con estructuras distintas (`data.sourceModule`, `affectedSections`, `closureType`)	M3	🟡 Esperando a M3 el valor de `data.sourceModule` que recibe M7. Anota además pendientes del Core: si el schema valida `data` o el evento completo, `ticketCreated` y el casing de un evento de M1
Validación formal de los schemas de `workOrderScheduled` y `workOrderCompleted`	M3	🟡 Cruzados por lectura el 29/10, compatibles y sin cambios de código. La prueba `requestId` → `sourceRequestId` se hizo el 5/11 y cerró bien. Falta la validación formal contra Draft 2020-12
Contrato final de cortes (28/10). M7 exige tramos `{street, from, to, direction}` y `closureType`	M7	🟡 Implementado y mergeado (#447). Falta que M7 declare `streetClosureEnded` en el Core y la prueba E2E entre M7 y el Core
Decisiones del Core sin tomar (28/10): validación de `format` en `data`, exigir `user_id` por AMQP, clave de barrio (UUID o slug) y quién es dueño de `updateTicketStatus`	M9 y M2	🟡 Esperando decisión. Nuestro código ya es compatible con la identidad por AMQP y seguimos con el UUID de barrio
Búsqueda de establecimientos. El acta no se deriva sin `establishmentId`	M4	🟡 Pedida por dirección, CUIT o barrio. Su contrato v1.2 RC (2/11) no la menciona y dice que aún no tienen publisher ni consumer operativos. Plan B de M6: el `establishmentId` se carga a mano al emitir el acta. Sin respuesta al mensaje del 5/11

Cerrados desde el 5 de octubre

Punto	Con quién	Cómo quedó
✅ **Quién emite la identidad**	M1	M1 emite el JWT de usuario (HS256, `iss` `modulo1-ciudadanos`, 15 minutos) y M6 lo valida; no emite tokens propios. Login real verificado de punta a punta el 3/11, con 12 cuentas de demostración. El rol de M1 es único y sin área: los roles dentro de M6 se asignan localmente (ADR-008). M6 consulta el directorio de empleados de M1 para validar cuadrillas
✅ **Bus de eventos**	M9	El bus es RabbitMQ del Core, en AWS: exchange `fanout` `muni.inbox`, cola `q.ambiente` creada por el Core y sobre propio del Core. M9 habilitó la red para Render el 28/10 y el módulo figura conectado. M6 tiene registradas 8 suscripciones y 6 publicaciones, y la prueba de punta a punta con M3 del 5/11 completó el ciclo
✅ **Catálogo de barrios**	M9	Publicado el 27/10: 48 barrios en 15 comunas. M6 lo adoptó con una zona operativa por barrio (#376), y el UUID viaja en `location.neighborhoodId`
✅ **Zona contra barrio**	M9	M9 aclaró el 20/10 que el Core no tiene ninguna «zona»: el barrio es de M9 y la zona operativa es nuestra, y la referencia por `neighborhoodIds[]`

Punto	Con quién	Cómo quedó
✅ `ticketUpdated / ROUTED` y `updateTicketStatus`	M2	Contrato cerrado el 26/10 con el Acuerdo y los Sobres v2.2, sobre su v1.78 (WIP). La fecha agendada va como texto en `publicMessage`. M2 confirmó que no consume `urbanServiceScheduled` ni `treePruningScheduled`
✅ **Contrato con M3**	M3	Hacia M3, solo `infrastructureRepairRequested`, `containerDamaged` y `treeRiskDetected`; de M3, solo `workOrderScheduled` y `workOrderCompleted`, correlacionados por `sourceRequestId`
✅ **Identificador del acta y resolución**	M4	`sourceViolationId` viaja en `commercialFineGenerated` y `closureUpdate`; `decidedAt` y `externalRef` confirmados el 27/10, `FORMAL_NOTICE` aceptado en `suggestedAction` y ruteo de `commercialFineGenerated` hacia nosotros confirmado el 21/10. El caso «M4 decide no accionar» no dispara ningún evento: lo cerramos por vencimiento de un plazo configurable
✅ **Respuestas de corte**	M7	M7 solo consume `streetClosureRequested`. Devuelve `closureRequestId` y `requestingModule` en las tres respuestas, y lee `data.sourceModule: "M6"`